Luleå Technical University
School of Computer Science, Electrical and Space Engineering
Luleå, Sweden

Project Proposal
Group 14
D7065E, Embedded Intelligence at the Edge

FIRE DETECTION AND AUTONOMOUS RESPONSE

Ekpenyong Esu Ekpenyong
ekpekp-5@student.ltu.se

Examiner: Johan Kristiansson
LTU, Luleå

21st September 2026

Repository: https://github.com/TEAM/d7065e-firelab

This proposal is provisional. Its purpose is to agree a direction, not to fix the design; several of the choices below are expected to change as the system is built.

1. Use case, and why

This project detects fires on a BuildSim office floor and responds to them without a human in the loop. It was chosen for two reasons.

First, the detection problem is hard in an interesting way. A smoke detector alone cannot tell a fire from a frying pan, a dusty corridor, or a failed sensor. Separating them requires fusing smoke, carbon monoxide and temperature across neighbouring rooms, which makes the autonomous part worth building rather than a single threshold.

Second, the response contains a real conflict. Closing a fire door contains the smoke but removes that room's escape route. The system has to trade containment against evacuation as the fire grows, so the decision is not merely a setpoint.

BuildSim supports this use case directly: it renders fire, smoke and sprinkler effects, stores door and lock state, computes shortest paths on a walkable graph that excludes locked doors, and displays room highlights, alerts and routes.

2. Scenario

A fire starts in office A109. At the same time somebody begins cooking in the kitchen A111, which produces visible smoke *first*. The system ignores the kitchen, because heat and CO never corroborate its smoke. In A109 all three modalities rise together, the alarm state machine advances through a verification delay to CONFIRMED, the sprinkler activates, and the fire is suppressed so that the measured values fall again. Fire doors unlock, smoke spreads towards A110, escape routes are recomputed around the affected rooms, and the operator sees red, orange and green room highlights with escape paths drawn on the BuildSim 3D view.

3. What will be built (rough sketch)

Each item below runs as its own process in a container and communicates through defined interfaces — MQTT between processes, and BuildSim's REST API for state.

- **Scenario generator** — starts fires and nuisance events, and publishes ground-truth labels so the detector can be trained and scored.
- **Fire physics** — owns the true heat, smoke and CO in every room, and spreads them between rooms.
- **Sensors** — emulated smoke, CO and temperature detectors that observe that truth imperfectly and can be made to fail.
- **Data pipeline and storage** — turns readings into features and keeps the history used for training and for dashboards.
- **Detector** — estimates the probability of fire in each room.
- **Decision agent** — runs the alarm state machine and issues commands; it never writes actuator state itself.
- **Actuators** — sprinklers and fire doors; validates every command against safety interlocks before applying it and reporting state back.
- **Evacuation and routing** — plans escape paths and moves occupants along them.
- **Visualisation** — highlights, alerts and route display on BuildSim; device history and dashboards in ThingsBoard.

The loop closes because the physics reads actuator and door state back from BuildSim, so a sprinkler command changes the next simulated measurement.

4. Simulating the building

BuildSim provides the rooms but no physics, so both the fire and the people are simulated.

Physical processes. Each room holds a true temperature, smoke density and CO concentration. A fire source grows with the standard t^2 law; heat, smoke and CO then relax towards a source-driven equilibrium with the room's own time constants. Smoke and heat move between rooms along BuildSim's room adjacency graph, scaled by door state, so a closed fire door slows the spread. Sprinkler activation reduces the heat release rate, which is how actuator actions influence future readings.

Nuisance sources. Cooking, dust, steam and smouldering material are modelled with their own signatures — deliberately, because a fire is separable from cooking only by the pattern across smoke, CO and heat together.

Sensors. Readings are not the truth. Each device applies a response lag, a calibration bias drawn once per device, noise, quantisation, range clamping and an optional injected fault, and reports only at its own sampling interval. Two detectors in the same room therefore disagree, as real ones do.

Movement of people. A weekday schedule drives normal occupancy — arrivals 07:00–09:00, lunch 11:30–13:00, departures 16:00–18:00, meeting rooms filling on booking, quiet at night — with randomness so that no two days match. During an incident, occupants move along routes returned by BuildSim's navigation graph.

The models are deliberately simple; what matters is realistic trends and a closed loop.

5. The autonomous part

The autonomous element decides whether a fire exists and what to do about it.

Detection starts as a transparent fusion rule over hand-built features: smoke level, its rate of rise, temperature rise above baseline, CO level, the CO-to-smoke ratio, and whether neighbouring rooms agree. That baseline is then compared against an isolation forest trained offline on scenario data, so the report can show whether the model actually beats the rule.

Response is an explicit state machine — NORMAL, INVESTIGATING, PRE-ALARM, CONFIRMED, SUPPRESSED — with dwell times between states. The verification delay is what suppresses false alarms, and the named states make the agent explainable rather than a black box. Sophistication is not the goal; starting simple is.

Safety rules sit in the actuator service and cannot be overridden by the agent: sprinklers require heat corroboration and not smoke alone, fire doors fail unlocked, and a room on an active escape route may not be sealed.

6. Biggest unknowns & risks

- Whether enough labelled scenario data can be generated to train a model that measurably beats the simple fusion rule. If not, the rule stands and the comparison itself becomes the result.
- Whether the fire physics is realistic enough for the decisions to mean anything. It is a lumped per-room model, not a CFD simulation.
- Whether the containment-versus-evacuation policy can be tuned sensibly, or whether it degenerates into always favouring evacuation.
- ThingsBoard is new to the team and heavier than the alternatives, and its dashboards are configured through a UI rather than written as code. Fallback: InfluxDB with Grafana.

- Keeping roughly a dozen processes synchronised in simulated time, and keeping BuildSim writes at a sane rate as sensors are added.
- Whether injected sensor faults are distinguishable from real events at all, which is the hardest of the classification cases.

7. First step & rough plan

The riskiest assumption is that the control loop closes at all — that a command issued by the agent changes a later sensor reading. That is prototyped first, in one room: fire physics → one sensor → detector → agent → sprinkler → back into the physics.

A pure-Python prototype of this loop has already been written and runs. It confirms a fire in the burning room 8 s after that room becomes hazardous, ignores a concurrent cooking event in the kitchen, and shows the true smoke curve bending once suppression begins.

Weeks 2–3	One room end-to-end against a live BuildSim; fire and smoke visible in the 3D view.
Weeks 3–4	Emulated sensors with lag, noise and faults; several rooms; spread across the adjacency graph.
Weeks 4–5	Scenario generator, data pipeline and ThingsBoard storage; a labelled dataset accumulating.
Weeks 5–6	Detector trained offline and compared against the baseline rule; actuators and safety interlocks.
Weeks 6–7	Evacuation routing, occupant movement, and the containment trade-off.
Week 8	Evaluation — detection latency, false alarms per day, missed detections, evacuation time — and the demo.